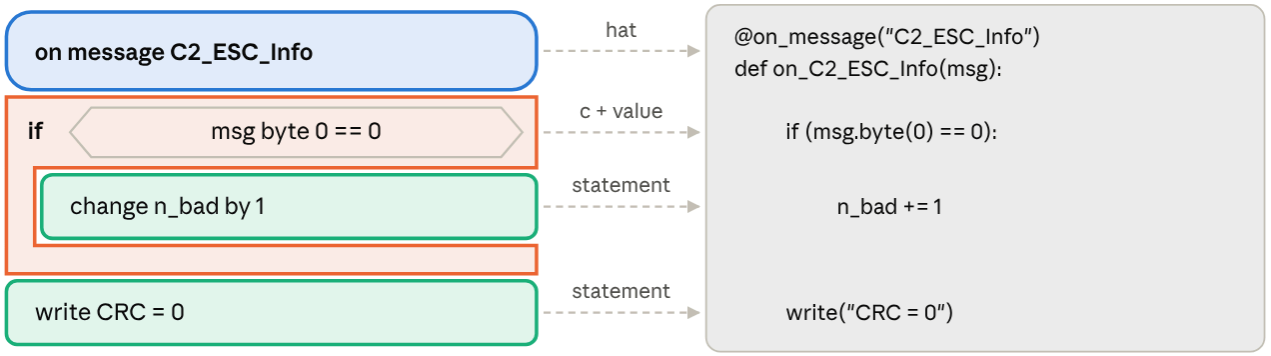
Hướng dẫn các block của Program

Oct 13, 2026 · @SangNguyen

Tổng quan

Mỗi stack trên tab Block là một hàm trong tab Code: block đầu (hat) là decorator và dòng `def`, các block ghép bên dưới là thân hàm. Hai tab là cùng một chương trình, đọc file nào cũng ra một kết quả.

Một stack trên tab Block là một hàm trong tab Code



Block value trong ô lục giác là một phần của dòng if, không thành dòng riêng.

stack → hàm · 4 block, 5 dòng code

Hat thành decorator và dòng `def`; block trong miệng của `if` thụt vào một mức; block nối dưới `if` quay về mức của `if`.

Hình dạng	Vai trò	Ví dụ
hat	Mở đầu stack: một handler hoặc một hàm. Chỉ nối xuống dưới	`on start`, `on message`, `define`
statement	Một dòng code. Nối cả trên và dưới	`send`, `set timer`, `write`
c (có miệng)	Chứa thân lệnh. `if else` có hai miệng	`if`, `repeat`, `while`
cap	Kết thúc một nhánh. Chỉ nối phía trên	`break`
value	Trả về giá trị, cắm vào ô của block khác: tròn = số, lục giác = điều kiện	`msg byte`, `a > b`, `Msg . Sig`

Cách thao tác:

- **Thêm block:** kéo từ Palette vào canvas, hoặc double-click trong Palette.

- **Nối block:** thả sát dưới block khác cho tới khi nó hít vào (snap). Value block thả vào ô tròn hoặc ô lục giác.
- **Xoá block:** kéo block về vùng Palette bên trái.
- **Sửa ô:** click vào ô trên block, hoặc dùng panel Property bên phải. Panel này còn có ô ghi chú; ghi chú thành comment `# ...` cuối dòng code.
- **Ô message:** gõ tên DBC (`C2_ESC_Info`), CAN ID hex (`23A`), hoặc `CAN2::C2_ESC_Info` để chỉ rõ network. Nút ▾ liệt kê message và signal của DBC.

Đồng bộ hai tab:

- Stack đổi ở tab Block được ghi sang tab Code khi mở tab Code, bấm Run, Validate hoặc Save. Lần ghi này là một bước Undo trong tab Code.
- Hàm sửa ở tab Code quay về tab Block khi mở lại tab Block. Code mà block không biểu diễn được hiện thành block Python màu xám, không mất gì.
- Tên hàm tự đặt theo mẫu: `on message C2_ESC_Info` thành `def on_C2_ESC_Info(msg):` , `on timer t1` thành `def on_timer_t1():` .
- Block năm rời, không dưới một hat nào, không bao giờ được ghi vào code và không chạy.
- Nhóm Diagnostics (UDS), Test Sequence và Python chỉ chạy băng Run, không xuất sang CAPL. Khi chọn target CAPL, các nhóm này bị làm mờ trong Palette.

Block sự kiện (hat)

Mỗi chương trình bắt đầu từ các hat: chúng nói *khi nào* stack chạy. Các handler chạy lần lượt từng cái, như CAPL, nên mỗi handler phải xong nhanh. Riêng `sequence` chạy trong luồng riêng và được phép chờ.

Block	Code sinh ra (ví dụ C2)	Chạy khi nào, cách xử lý
`on start`	`@on_start` / `def start():`	Một lần sau khi bấm Run. Đặt giá trị ban đầu, bật `send cyclic` , đặt timer đầu tiên
`on stop`	`@on_stop` / `def stop():`	Khi bấm Stop hoặc HOME › Stop. Ghi tổng kết vào Log

Block	Code sinh ra (ví dụ C2)	Chạy khi nào, cách xử lý
`on message {msg}`	`@on_message("C2_IVC_CVC_StsEpt")` / `def on_C2_IVC_CVC_StsEpt(msg):`	Mỗi frame nhận được của message đó (frame mình gửi không tính). Trong thân dùng `msg byte`, `msg signal`, `msg id`, `forward`. Gõ `CAN1::*` để nhận mọi frame CAN1 chưa có handler riêng (gateway)
`on signal {msg} . {sig}`	`@on_signal("C2_IVC_CVC_StsEpt", "C2_IVC_CVC_BrkPedalPct")` / `def on_..._C2_IVC_CVC_BrkPedalPct(value):`	Chỉ khi giá trị signal **đổi**. `value` là giá trị vật lý mới. Hai signal của cùng một message: dùng một `on message` đọc cả hai
`on timer {name}`	`@on_timer("t_monitor")` / `def on_timer_t_monitor():`	Khi timer hết giờ. Timer chỉ chạy một lần: muốn lặp thì đặt `set timer` lại ngay trong stack này
`on key {key}`	`@on_key("c")` / `def on_key_c():`	Khi bấm phím lúc đang Run và cửa sổ Program có focus. Nhận `a`..`z`, `0`..`9`, `F1`..`F12`; `*` là mọi phím. Dùng để tiêm lỗi bằng tay
`define {name}`	`def tinh_toc_do():`	Hàm dùng chung, gọi bằng block `call`. Không có tham số; cần tham số thì dùng hat Python `python def set_speed(kmh):`
`sequence {name}`	`@sequence("ESC scenario")` / `def seq_ESC_scenario():`	Kịch bản chạy sau `on start`, theo thứ tự trong file, trong luồng riêng. Chỉ ở đây mới dùng được `wait`, `expect`, UDS. Python only

Ví dụ: ESC giả trả lời nút ESC OFF của MHU. Ghép trên tab Block:

```
on start
├ set  C2_ESC_Info . C2_STAT_DSC  to  ESC normal
└ send  C2_ESC_Info  every  20  ms

on message  C2_MHU_CarMode_Switch
└ if  ‹ msg signal C2_MHU_ESCOFF_Sw  ==  2 ›
    └ set  C2_ESC_Info . C2_STAT_DSC  to  ESC-OFF
```

Code sinh ra trong tab Code (ngoặc quanh điều kiện là do block viết, không sao):

```
@on_start
def start():
    set_signal("C2_ESC_Info", "C2_STAT_DSC", "ESC normal")
    send_cyclic("C2_ESC_Info", 20)


@on_message("C2_MHU_CarMode_Switch")
def on_C2_MHU_CarMode_Switch(msg):
    if (msg.signal("C2_MHU_ESCOFF_Sw") == 2):
        set_signal("C2_ESC_Info", "C2_STAT_DSC", "ESC-OFF")
```

Lưu ý khi xử lý:

- Mỗi loại sự kiện nên có một hat. Hai hat cùng sự kiện thì cả hai đều chạy, và tab Issue báo Warning "Second handler".
- Handler không được chờ. Cần "làm A, 800 ms sau làm B" thì làm A rồi `set timer`, B nằm trong `on timer`.
- Muốn `on message` nhận cả frame mình gửi ra, sửa ở tab Code: `@on_message("Msg", tx=True)`.

Block CAN & DBC

Nhóm CAN gửi, đọc và chuyển tiếp frame. Mỗi message DBC có một ảnh TX (giá trị các signal): `set` sửa ảnh này, `send` và `send ... every` đưa ảnh ra bus, CRC và Alive được điền tự động khi DBC đánh dấu chúng (comment Checksum / Alive counter) hoặc khi có khai báo e2e_message.

Block	Code sinh ra (ví dụ C2)	Cách xử lý
`send {id} data {data}`	`send(0x7DF, [0x02, 0x3E, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00])`	Frame thô theo CAN ID, byte gõ dạng hex `02 3E 00`. Block không có ô network: khi nối nhiều network, sửa ở tab Code thêm `channel="CAN1"`
`send {msg}`	`send("C2_ESC_Info")`	Gửi một frame từ ảnh TX: các `set` trước đó, rồi Alive và CRC
`set {msg} . {sig} to {value}`	`set_signal("C2_ESC_VAFS", "C2_EPB_Sts", "Locked")`	Sửa ảnh TX. Giá trị là số vật lý (`12.5` km/h) hoặc chữ trong bảng giá trị DBC (`Locked`). Ra bus ở frame chu kỳ kế tiếp hoặc lần `send` sau
`send {msg} every {ms} ms`	`send_cyclic("C2_ESC_ABS_Status", 20)`	Gửi theo chu kỳ. Gọi lại với chu kỳ khác thì chỉ đổi chu kỳ. Muốn lấy chu kỳ của DBC: viết `send_cyclic("Msg")` ở tab Code
`stop cyclic {msg}`	`stop_cyclic("C2_ESC_ABS_Status")`	Dừng gửi chu kỳ, ví dụ để thử timeout. Bật lại bằng `send ... every`
`forward msg to {channel}`	`forward(msg, "CAN2")`	Chỉ đặt dưới `on message`. Gửi frame vừa nhận sang network khác, nguyên vẹn: ID, DLC, dữ liệu, CRC, Alive
`msg id` (value)	`msg.id`	CAN ID của frame vừa nhận, dùng trong `on message CAN1::*` để lọc
`msg byte {n}` (value)	`msg.byte(0)`	Byte thứ n của frame vừa nhận. Với C2, byte 0 là CRC
`msg signal {sig}` (value)	`msg.signal("C2_MHU_ESCOFF_Sw")`	Giá trị vật lý của signal trong frame vừa nhận

Block	Code sinh ra (ví dụ C2)	Cách xử lý
`{msg} . {sig}` (value)	`get_signal("C2_ESC_ABS_Status", "C2_ESC_VehicleSpeed")`	Giá trị mới nhất trên bus, kể cả frame mình gửi. Chưa có frame nào thì trả về start value của DBC

Ví dụ 1: ESC giả bật đèn phanh theo bàn đạp phanh của IVC. `value` là block biến (nhóm Variables) có tên `value`.

```
on start
├ set  C2_ESC_ABS_Status . C2_ESC_VehicleSpeedValid  to  Valid
├ send  C2_ESC_ABS_Status  every  20  ms
└ send  C2_ESC_VAFS  every  40  ms

on signal  C2_IVC_CVC_StsEpt . C2_IVC_CVC_BrkPedalPct
└ if  ‹ value  >  5 ›
    | set  C2_ESC_VAFS . C2_ESC_BrkLampReq  to  1
  else
    | set  C2_ESC_VAFS . C2_ESC_BrkLampReq  to  0
```

```
@on_signal("C2_IVC_CVC_StsEpt", "C2_IVC_CVC_BrkPedalPct")
def on_C2_IVC_CVC_StsEpt_C2_IVC_CVC_BrkPedalPct(value):
    if (value > 5):
        set_signal("C2_ESC_VAFS", "C2_ESC_BrkLampReq", 1)
    else:
        set_signal("C2_ESC_VAFS", "C2_ESC_BrkLampReq", 0)
```

Ví dụ 2: gateway hai chiều ESC (CAN1) và IVC (CAN2), chỉ cần 4 block:

```
on message  CAN1::*
└ forward msg to  CAN2

on message  CAN2::*
└ forward msg to  CAN1
```

Những việc chưa có block riêng: viết trong block Python xám (nhóm Python) hoặc ở tab Code.

Code	Dùng để
`forward(msg, "CAN2", {"C2_ESC_VehicleSpeed": 120})`	Chuyển tiếp có sửa signal: CRC tính lại, Alive giữ như ECU gửi

Code	Dùng để
`e2e_config(...)`, `e2e_message(...)`	Khai báo E2E, đặt ở cấp ngoài cùng của file. Nút **E2E…** viết dòng này giúp
`e2e_enable("C2_ESC_ABS_Status", crc=False)`	Tắt CRC hoặc Alive để tiêm lỗi; `True` để bật lại
`set_generator("Msg", "Sig", "Ramp", 0.5)`	Generator như Send CAN: `None`, `CheckSum`, `Alive Counter`, `Ramp`, `Random`
`get_signal("Msg", "Sig", rx_only=True, max_age=500)`	Chỉ lấy frame do node khác gửi, không cũ quá 500 ms; không có thì trả `None`
`msg_age("Msg", rx_only=True)`, `last_frame("Msg")`	Tuổi frame cuối (ms) để giám sát timeout; frame cuối để đọc `data`

Block timer, chờ và kiểm tra

Timer dùng trong handler để "làm việc sau"; `wait`, `wait for`, `expect` chỉ dùng trong `sequence` vì chúng chờ. Đặt chúng trong handler thì tab Issue báo Error và Run từ chối chạy.

Block	Code sinh ra (ví dụ C2)	Cách xử lý
`set timer {name} to {ms} ms`	`set_timer("t_epb", 800)`	Đặt hoặc đặt lại timer chạy một lần. Hết giờ thì stack `on timer t_epb` chạy. Muốn lặp: đặt lại trong chính `on timer`
`cancel timer {name}`	`cancel_timer("t_epb")`	Huỷ timer đang chờ; `on timer` sẽ không chạy
`wait {ms} ms`	`wait(1000)`	Chỉ trong `sequence`. Kịch bản dừng lại, các handler vẫn chạy bình thường
`{var} = wait for {msg} max {ms} ms`	`m = wait_for_message("C2_ESC_Info", timeout=500)`	Chỉ trong `sequence`. Lấy frame nhận kế tiếp của message; hết giờ thì `m` là `None`. Kiểm tra băng block điều kiện Python xám `m is not None`

Block	Code sinh ra (ví dụ C2)	Cách xử lý
`expect {msg} . {sig} = {value} within {ms} ms`	`expect_signal("C2_ESC_Info", "C2_STAT_DSC", "ESC-OFF", timeout=1000)`	Chỉ trong `sequence`. Chờ signal đạt giá trị, ghi `OK` hoặc `FAILED` kèm thời gian vào Log. Muốn dùng kết quả True/False để đếm PASS/FAIL: viết `ok = expect_signal(...)` ở tab Code

Ví dụ 1: nhấn công tắc EPB, ESC báo Releasing rồi 800 ms sau báo Released. Handler không chờ mà đặt timer:

```
on signal  C2_IVC_CVC_HV_DrvSys_status . C2_IVC_CVC_EPB_SwitchSta
└ if  ‹ value  ==  1 ›
    ├ set  C2_ESC_VAFS . C2_EPB_Sts  to  Releasing
    └ set timer  t_epb  to  800  ms

on timer  t_epb
└ set  C2_ESC_VAFS . C2_EPB_Sts  to  Released
```

Ví dụ 2: timer lặp 100 ms để giám sát. Stack tự đặt lại timer ở dòng cuối:

```
on start
└ set timer  t_mon  to  100  ms

on timer  t_mon
├ write  ‹ C2_IVC_CVC_StsEpt . C2_IVC_CVC_BrkPedalPct ›
└ set timer  t_mon  to  100  ms
```

Ví dụ 3: sequence thử ESC khi mất message của IVC. Giá trị mong đợi chỉ là ví dụ, đổi theo spec ESC:

```
sequence  ESC timeout test
├ stop cyclic  C2_IVC_CVC_StsEpt
├ wait  2000  ms
├ expect  C2_ESC_Info . C2_ESC_Fault_Status  =  1  within  500  ms
├ send  C2_IVC_CVC_StsEpt  every  20  ms
└ expect  C2_ESC_Info . C2_ESC_Fault_Status  =  No Fault  within  1000  ms
```

```
@sequence("ESC timeout test")
def seq_ESC_timeout_test():
    stop_cyclic("C2_IVC_CVC_StsEpt")
    wait(2000)
    expect_signal("C2_ESC_Info", "C2_ESC_Fault_Status", 1, timeout=500)
    send_cyclic("C2_IVC_CVC_StsEpt", 20)
    expect_signal("C2_ESC_Info", "C2_ESC_Fault_Status", "No Fault",
timeout=1000)
```

Block điều khiển, biến, phép toán, hàm, Log và Python

Những block này là logic chung của chương trình. Quy tắc cắm: ô tròn nhận số, ô lục giác chỉ nhận điều kiện (so sánh, `and`, `not`), ô `any` nhận mọi thứ.

Nhóm	Block	Code sinh ra	Cách xử lý
Control	`if {cond}`	`if (speed > 0.5):`	Cắm một điều kiện, kéo các block cần chạy vào miệng
Control	`if {cond}` … `else`	`if …:` / `else:`	Hai miệng: miệng trên khi đúng, miệng dưới khi sai
Control	`repeat {n} times`	`for _ in range(10):`	Lặp n lần. Trong `sequence`, đặt `wait` trong thân để lặp theo thời gian
Control	`while {cond}`	`while (n < 10):`	Lặp khi điều kiện đúng. Trong handler, vòng lặp không dừng sẽ bị runtime cắt và báo lỗi
Control	`break`	`break`	Thoát vòng lặp gần nhất. Không nối được gì bên dưới
Variables	`set {var} to {value}`	`fake_speed = 1`	Gán biến. Biến toàn cục không cần `global`, runtime tự thêm
Variables	`change {var} by {value}`	`n_fault += 1`	Cộng dồn. Biến khai báo kiểu `cnt: byte = 0` tự tràn như C (255 + 1 = 0)
Variables	`{var}` (value)	`n_fault`	Đọc biến. Không cắm thẳng vào ô lục giác được; dùng `n_fault > 0` hoặc block điều kiện Python

Nhóm	Block	Code sinh ra	Cách xử lý
Operators	`{a} + {b}`	`(a + b)`	`+ - * / % //`; `/` ra số thực, `//` chia lấy nguyên
Operators	`{a} == {b}`	`(a == b)`	`== != < <= > >=`, ra điều kiện
Operators	`{a} and {b}`, `not {a}`	`(a and b)`, `(not a)`	Ghép điều kiện
Operators	`{a} & {b}`	`(a & b)`	`&`, `|`, `^`, `<<`, `>>`: xử lý bit, ví dụ `msg byte 1 & 15` lấy 4 bit thấp
Operators	`bit {n} of {x}`	`((x >> n) & 1)`	Lấy một bit, ra 0 hoặc 1
Functions	`call {name}`	`esc_mac_dinh()`	Gọi hàm tạo bằng hat `define`
Log	`write {text}`	`write("ESC về mặc định")`	Ghi một dòng vào tab Log, kèm thời gian từ lúc Run. Cắm được value block để in giá trị
Python	`python {code}`	như đã gõ	Một hoặc nhiều lệnh Python bất kỳ
Python	`{code}` (value / điều kiện)	như đã gõ	Biểu thức hoặc điều kiện Python, ví dụ `m is not None`
Python	`python {head}` (c)	`for i in range(3):`	Dòng `for` / `with` có thân là các block
Python	`python {head}` (hat)	`def set_speed(kmh):`	Handler hoặc hàm có tham số; block bên dưới là thân
Python	`python code {code}` (hat)	cả hàm	Cả một hàm viết bằng Python, không nối gì bên dưới
Comment	`# {text}`	`# ghi chú`	Một dòng comment trong code

Ví dụ 1: đếm số frame `C2_ESC_Info` báo lỗi, cứ 50 frame ghi Log một lần (if lồng if, phép `%`, biến):

```
on message  C2_ESC_Info
└ if  ‹ msg signal C2_ESC_Fault_Status  !=  0 ›
    ├ change  n_fault  by  1
    └ if  ‹ ‹ n_fault % 50 ›  ==  1 ›
        └ python  write("ESC báo lỗi %d frame", n_fault)
```

```
@on_message("C2_ESC_Info")
def on_C2_ESC_Info(msg):
    if (msg.signal("C2_ESC_Fault_Status") != 0):
        n_fault += 1
        if ((n_fault % 50) == 1):
            write("ESC báo lỗi %d frame", n_fault)
```

Biến `n_fault` cần giá trị đầu: thêm bằng nút + ở tab Var & Time (`n_fault = 0`), hoặc viết dòng đó ở đầu tab Code.

Ví dụ 2: một hàm dùng lại ở hai nơi (`define` + `call`):

```
define  esc_mac_dinh
├ set  C2_ESC_Info . C2_STAT_DSC  to  ESC normal
└ set  C2_ESC_Info . C2_ESC_Fault_Status  to  No Fault

on start
└ call  esc_mac_dinh

on key  r
├ call  esc_mac_dinh
└ write  ESC về mặc định
```

Ví dụ 3: biến đúng/sai làm điều kiện, và việc chưa có block riêng viết bằng block Python xám:

```
on message  CAN1::C2_ESC_ABS_Status
└ if  ‹ fake_speed ›                      ← block điều kiện Python
    | python  forward(msg, "CAN2", {"C2_ESC_VehicleSpeed": 120})
  else
    | forward msg to  CAN2
```

Block UDS (Diagnostics)

Nhóm UDS gửi request chẩn đoán tới một ECU của Diagnostic Setting và đọc phản hồi. Tất cả là Python only và đặt trong `sequence` , vì request phải chờ ECU trả lời. ECU (ví dụ `ESC`) phải có trong Diagnostic Setting: ID request / response, kênh Diagnostic, DLL Security Access.

Block	Code sinh ra (ví dụ ESC)	Cách xử lý
`{var} = UDS {ecu} {req}`	`resp = uds_request("ESC", "22 F1 90")`	Gõ request dạng hex có dấu cách. Kết quả nằm trong biến `resp`, Log ghi request và phản hồi
`security access {ecu} level {level}`	`security_access("ESC", 1)`	`27 01` lấy seed, tính key bằng DLL của ECU, gửi `27 02`. Mở khoá xong thì ECU cho phép dịch vụ bảo vệ
`tester present {ecu} on / off`	`tester_present("ESC", True)`	Gửi `3E 80` mỗi 2 s khi không có request, giữ phiên extended. Tắt trước khi kết thúc test
`{var} positive` (điều kiện)	`resp.positive`	Đúng khi phản hồi dương. Căm vào `if`
`{var} byte {n}` (số)	`resp.byte(3)`	Byte thứ n của phản hồi, tính từ SID dương: với `62 F1 86 01`, byte 0 là `0x62`, byte 3 là phiên hiện tại

Phần còn lại của kết quả chưa có block, dùng block value Python xám:

Code	Giá trị
`resp.nrc`	Mã NRC khi phản hồi âm, ví dụ `0x31` cho `7F 22 31`
`resp.hex()`	Cả phản hồi dạng chữ: `"50 03 00 32 01 F4"`
`resp.ascii(3)`	Phần từ byte 3 thành chữ, ví dụ VIN của `22 F1 90`
`resp.error`	Lý do khi không có phản hồi (timeout, ECU không có trong Diagnostic Setting …)

Ví dụ: vào phiên extended, mở khoá, đọc VIN, kiểm tra phiên rồi trả ESC về bình thường:

```
sequence  ESC read VIN
├ resp = UDS  ESC  10 03
├ tester present  ESC  on
├ security access  ESC  level  1
├ resp = UDS  ESC  22 F1 90
├ if  ‹ resp positive ›
|   | python  write("VIN %s", resp.ascii(3))
| else
|   | python  write("22 F1 90 FAILED: %s", resp.hex() or resp.error)
├ resp = UDS  ESC  22 F1 86
├ if  ‹ resp byte 3  ==  3 ›
|   | write  extended session OK
├ tester present  ESC  off
└ resp = UDS  ESC  10 01
```

```
@sequence("ESC read VIN")
def seq_ESC_read_VIN():
    resp = uds_request("ESC", "10 03")
    tester_present("ESC", True)
    security_access("ESC", 1)
    resp = uds_request("ESC", "22 F1 90")
    if resp.positive:
        write("VIN %s", resp.ascii(3))
    else:
        write("22 F1 90 FAILED: %s", resp.hex() or resp.error)
    resp = uds_request("ESC", "22 F1 86")
    if (resp.byte(3) == 3):
        write("extended session OK")
    tester_present("ESC", False)
    resp = uds_request("ESC", "10 01")
```

Kiểm tra phản hồi âm mong đợi: sau `resp = UDS ESC 22 FF FF`, cắm điều kiện Python `not resp.positive and resp.nrc == 0x31` vào `if`. Mẫu đầy đủ có đếm PASS / FAIL nằm trong file `c2_esc_uds_test.py`.

Khi bus là CAN FD

Các block vẫn giữ nguyên; khác biệt nằm ở kiểu frame gửi ra. Frame gửi theo **tên DBC** là FD hay classic do DBC quyết định. Frame gửi theo **CAN ID** là FD nếu network đó bật chế độ FD. Chế độ FD và data bitrate của network đặt trong cấu hình kênh, không đặt trong Program.

Block	Trên network CAN FD
`send {id} data {data}`, `send {id} every` theo CAN ID	Luôn gửi dạng FD, bật BRS. Dữ liệu tối đa 64 byte. Độ dài không hợp lệ được đệm `00` lên 12, 16, 20, 24, 32, 48 hoặc 64 byte
`send {msg}`, `send {msg} every` theo tên DBC	Theo DBC: message khai báo FD thì gửi FD có BRS; message classic vẫn gửi classic. Gửi message FD lên network không bật FD thì Run báo lỗi `is a CAN FD message but network … is not in FD mode`
`set {msg} . {sig}`	Signal nằm ở bất kỳ đâu trong 64 byte
`on message {msg}`	Nhận cả frame classic lẫn FD
`msg byte {n}`	`n` từ 0 tới độ dài frame − 1, tối đa 63
`forward msg to {channel}`	Giữ kiểu frame và BRS như lúc nhận. Frame classic sang network FD vẫn là classic. Frame FD ⩽ 8 byte sang network classic thành classic. Frame FD > 8 byte sang network classic: Run báo lỗi trong handler
`wait for`, `expect`, `{msg} . {sig}`	Không đổi
Block UDS	Không đổi trong Program; frame UDS đi theo cấu hình kênh Diagnostic của ECU

Thuộc tính của frame nhận được, dùng trong block điều kiện hoặc value Python xám:

Code	Giá trị	Trong CAPL xuất ra
`msg.is_fd`	`True` nếu là frame FD	`this.FDF`
`msg.brs`	`True` nếu bật bit rate switch	`this.BRS`
`msg.dlc`	Số byte dữ liệu (8, 12, …, 64), không phải mã DLC	`this.DataLength`

Ví dụ 1: gửi một frame FD theo CAN ID. Gõ 10 byte, trên bus ra 12 byte vì 10 không phải độ dài FD hợp lệ. CAN ID và network chỉ là ví dụ:

```
send(0x1A0, [0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77, 0x88, 0x99, 0xAA],
channel="CANFD1")
# trên bus: 11 22 33 44 55 66 77 88 99 AA 00 00   (FD, BRS, 12 byte)
```

Ví dụ 2: gateway giữa một network FD (CAN2) và một network classic (CAN1). Frame FD dài hơn 8 byte không sang được bus classic nên bị bỏ và đếm lại:

```
on message  CAN2::*
└ if  ‹ msg.is_fd and msg.dlc > 8 ›          ← block điều kiện Python
    | change  n_dropped  by  1
  else
    | forward msg to  CAN1
```

```
@on_message("CAN2::*")
def on_CAN2_any(msg):
    if msg.is_fd and msg.dlc > 8:
        n_dropped += 1
    else:
        forward(msg, "CAN1")
```

Ví dụ 3: E2E trên frame FD 32 byte. Frame dài thường dùng CRC 16 bit có Data ID; CRC tính trên mọi byte còn lại của frame, kể cả byte đệm. Thuật toán và Data ID phải theo spec E2E của message:

```
e2e_message(0x1A0, network="CANFD1", crc="bytes0-1 big", alive="byte2 bit0-
3",
            algo="CRC-16/CCITT-FALSE", data_id=0x1A0, alive_range=(0, 14))

@on_start
def start():
    send_cyclic(0x1A0, 10, [0] * 32, channel="CANFD1")    # mỗi frame: Alive
rồi CRC-16
```

`bytes0-1 big` nghĩa là byte 0 giữ byte cao của CRC, byte 1 giữ byte thấp. Trong hộp thoại **E2E…**, chọn **CAN ID…** và đặt **Bytes** = 32 để lưới bit và bản xem trước đủ 32 byte; message FD trong DBC tự hiện đủ độ dài.

Một chỗ Run và Export CAPL khác nhau: khi Run trên network FD, frame gửi theo CAN ID dài ≤ 8 byte vẫn đi dạng FD; CAPL xuất ra gửi chúng dạng classic, chỉ frame > 8 byte mới có `FDF = 1`, `BRS = 1`. Muốn hai bên giống nhau, gửi theo tên DBC để DBC quyết định kiểu frame.

Quy tắc chung và lỗi thường gặp

Hầu hết lỗi khi ghép block hiện ngay ở tab Issue trước khi Run; double-click dòng lỗi để nhảy tới đúng chỗ.

Thông báo	Nguyên nhân	Cách sửa
`wait() waits, so it is only allowed inside @sequence`	Block chờ (`wait`, `wait for`, `expect`, UDS) nằm trong handler hoặc trong hàm `define`	Chuyển vào `sequence`. Trong handler thì dùng `set timer` + `on timer`
`message 'C2_ESC_Info' is in the DBC of several networks`	Hai network nạp cùng DBC, ô message chưa nói network nào	Gõ `CAN1::C2_ESC_Info`, hoặc chuột phải dòng lỗi » chọn cách sửa
`Second handler for …; both will run`	Hai hat cùng sự kiện, hoặc hai `on signal` trên cùng một message	Gộp vào một hat; hai signal cùng message thì dùng một `on message`
`Timer "t_epb" has a handler but set_timer(...) is never called`	Có `on timer` mà không có `set timer` nào đặt nó	Thêm `set timer` ở `on start` hoặc nơi bắt đầu công việc
`… is filled by the CheckSum generator on every frame` (lúc Run)	`set` vào signal CRC hoặc Alive đang được tính tự động	Không ghi tay. Muốn làm sai để thử thì dùng `e2e_enable(..., crc=False)`
Tên tham số trùng tên biến toàn cục	Hàm Python xám có tham số trùng một biến của chương trình	Đổi tên tham số, ví dụ `kmh` thay vì `speed`

Quy tắc khi ghép block:

1. Handler phải xong nhanh. Handler chạy quá 20 ms làm chậm mọi handler khác và Log cảnh báo một lần.
2. Mọi việc cần chờ đều đặt trong `sequence`. Các sequence chạy lần lượt theo thứ tự trong file, sau `on start`.
3. Block năm rời, không dưới hat nào, không bao giờ chạy.
4. Giá trị signal nên gõ bằng chữ của bảng giá trị DBC (`Valid`, `ESC-OFF`) để dễ đọc và không sai số.
5. Khai báo E2E (`e2e_message`) đặt ở cấp ngoài cùng của tab Code, dùng nút **E2E…** để viết.
6. Chương trình muốn xuất sang CAPL: chọn target CAPL và tránh nhóm Diagnostics (UDS), Test Sequence, Python. Export CAPL… liệt kê handler không xuất được và lý do.
7. Phần tuỳ chọn (kịch bản, test) đặt trong unit riêng; không cần thì tắt unit đó thay vì xoá block.